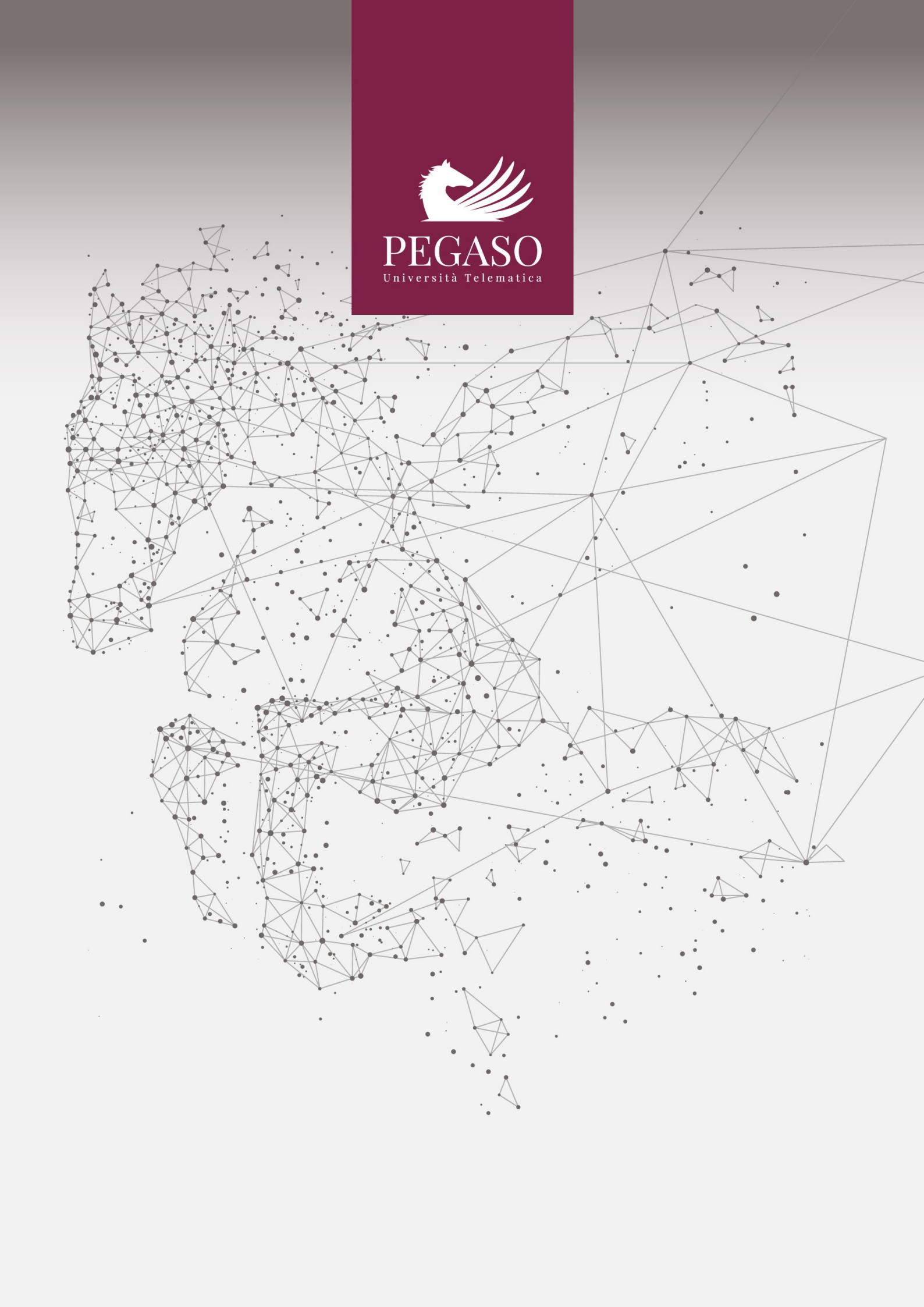

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. FUNZIONI E PROPRIETÀ DEGLI ARRAY IN NUMPY 4

3. GENERARE ARRAY 9

4. INDICIZZAZIONE E SLICING 11

5. OPERAZIONE TRA ARRAY 14

5.1 FUNZIONI VETTORIALI UNIVERSALI 15

BIBLIOGRAFIA 18

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si presentano le funzioni offerte dalla libreria NumPy che modificano le proprietà degli array o che eseguono operazioni sugli array stessi.

Vengono illustrate, inoltre, diverse funzioni per l'inizializzazione degli array secondo diversi criteri come ad esempio casualità, sequenzialità e uniformità. Si presentano, infine, le potenzialità dell'indicizzazione e dello slicing con gli array con NumPy, potenzialità maggiori rispetto a quanto offerto da Python con le liste.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Funzioni e proprietà degli array in Numpy

La funzione **copy**() offerta da NumPy permette di copiare gli elementi di partenza in un nuovo array. Qualsiasi modifica all'array di partenza non altererà la copia. La chiamata esplicita alla copia di un array si rende necessaria nei casi in cui l'array da copiare fosse molto numeroso (es. un miliardo di elementi). In questo caso, infatti, sarebbe opportuno fare delle considerazioni in merito a procedere o meno con la copia. Il formato di invocazione della funzione copy() è il seguente:

numeri_copia = numeri.copy()

Un aspetto molto interessante è che le proprietà **dtype** e **shape** degli array non sono in sola lettura. **dtype** può, infatti, essere modificata per modificare il tipo di elementi dell'array (che non è immutabile). La funzione che permette di cambiare tale tipo è la funzione **astype**() con il seguente formato:

astype(dtype, casting = "unsafe", copy = True)

È possibile però che nella conversione in un tipo di dati più specifico, alcune informazioni possano perdersi (es. da float ad int si perdere la parte decimale del numero). Di seguito si mostra un esempio di un array ottenuto tramite la funzione ones() (quindi formato da tutti elementi pari a 1.0) trasformato in un array di interi (con elementi tutti pari ad 1).

```
import numpy as np
vettore = np.ones(5)
print(vettore)
vettore2 = vettore.astype(np.uint64)
print(vettore2)
```

```
[1. 1. 1. 1. 1.]
[1 1 1 1 1]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In merito alla modifica della proprietà **shape**, invece, procederemo con il seguente esercizio:

- definizione di un array NumPy monodimensionale con i valori da 1 a 12;
- visualizzazione a schermo della proprietà shape;
- modifica della proprietà shape trasformando l'array in una matrice 2x6 (sempre da 12 elementi, ma bidimensionale) con l'istruzione v.shape = [2,6];
- visualizzazione a schermo nuovamente della proprietà shape.

```
import numpy as np
v = np.array(range(1,13), copy = True)
print("Array =",v)
print("Shape dell'array =", v.shape)
v.shape = [2,6]
print("Array dopo modifica forma =",v)
print("Shape dell'array dopo modifica forma =", v.shape)
```

```
Array = [ 1  2  3  4  5  6  7  8  9 10 11 12]
Shape dell'array = (12,)
Array dopo modifica forma = [[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]]
Shape dell'array dopo modifica forma = (2, 6)
```

Per modificare la forma di una matrice esiste anche la funzione **reshape**(). Essa risulta essere una funzione molto comoda per le matrici con 3 o più dimensioni. Infatti, invece di scrivere liste annidate, conoscendo il numero degli elementi che saranno contenuti nella matrice, si potrà procedere come segue (si noti che il prodotto delle dimensioni della matrice fornisce come risultato sempre il numero degli elementi dell'array di partenza, nel caso di esempio 12):

```
import numpy as np
v = np.array(range(1,13), copy = True)
matrice = np.array(v).reshape(2,3,2)
print(matrice)
```

```
[[[ 1  2]
  [ 3  4]
  [ 5  6]]

 [[ 7  8]
  [ 9 10]
  [11 12]]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Vediamo, ora, un esempio di uso combinato della funzione arange() con la funzione reshape() che può lanciare un'eccezione di tipo **ValueError** (prodotto delle nuove dimensioni diverso dal numero di elementi di partenza – nel caso mostrato 5*7=35 diverso da 48).

```
import numpy as np
v = np.arange(1,49)
m6x8 = v.reshape(6,8)
m12x4 = v.reshape(12,4)
m3x16 = v.reshape(3,16)
m_errato = v.reshape(5,7)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-16-a03c62a185c5> in <module>
      4 m12x4 = v.reshape(12,4)
      5 m3x16 = v.reshape(3,16)
----> 6 m_errato = v.reshape(5,7)
      7

ValueError: cannot reshape array of size 48 into shape (5,7)
```

La **trasposizione** di una matrice è "accessibile" attraverso una vista (valore dell'attributo T), come mostrato nel codice seguente:

```
import numpy as np
v = np.array(range(1,9), copy = True)
m = v.reshape(2,4)
print("Matrice originale")
print(m)
print()
print("Matrice trasposta")
print(m.T)
```

```
Matrice originale
[[1 2 3 4]
 [5 6 7 8]]

Matrice trasposta
[[1 5]
 [2 6]
 [3 7]
 [4 8]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'attributo T, infatti, mostra la matrice scambiando le dimensioni (orizzontale con verticale e viceversa). La funzione **swapaxes**(), invece, è una versione più generale di T poiché esegue la trasposizione di un array multidimensionale scambiando i due assi passati come parametri. Naturalmente passando gli assi 0 e 1 per un array bidimensionale si esegue semplicemente la trasposizione dell'array come visto in precedenza e come mostrato nell'esempio seguente:

```
import numpy as np
v = np.array(range(1,9), copy = True)
m = v.reshape(2,4)
print("Matrice originale")
print(m)
print()
print("Matrice trasposta")
print(m.swapaxes(0,1))
```

```
Matrice originale
[[1 2 3 4]
 [5 6 7 8]]

Matrice trasposta
[[1 5]
 [2 6]
 [3 7]
 [4 8]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

La funzione **transpose**() è una versione ancora più generale di swapaxes(). Essa permuta alcuni o tutti gli assi di un array multidimensionale sulla base del parametro fornito che deve essere una tupla. Nell'esempio seguente, il primo asse rimane «verticale», mentre gli altri due assi vengono scambiati (equivalente a swapaxes(1,2)):

```
import numpy as np
v = np.array(range(1,9), copy = True)
m = v.reshape(2,2,2)
print("Matrice originale")
print(m)
print()
print("Matrice trasposta")
print(m.transpose(0,2,1))
```

```
Matrice originale
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]

Matrice trasposta
[[[1 3]
  [2 4]]

 [[5 7]
  [6 8]]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Generare array

Gli array NumPy possono essere creati non solo mediante importazioni di strutture dati Python, ma anche in altri modi. Alcune funzioni sono state analizzate, ma ne esistono molte altre. Le inizializzazioni più importanti sono per:

- valori consecutivi (funzione arange());
- **valori casuali;**
- array vuoto (funzione empty());
- **stesso valore in ogni posizione** (zeros(), ones(), ecc.);
- **valori uniformemente distribuiti in un intervallo.**

Molto utile per la gestione di test ed esperimenti è la generazione di un **array di valori casuali**. Per fare ciò è possibile sfruttare il modulo di NumPy dedicato alla gestione di valori pseudocasuali **numpy.random**. Nell'esempio seguente, si invoca la funzione **random**() di numpy.random che permette di ottenere una matrice 3x3 (le dimensioni sono state passate come tupla alla funzione random()).

```
import numpy as np
m_casuale = np.random.random((3,3))
print(m_casuale)
```

```
[[0.73663631 0.56138011 0.84386645]
 [0.05860862 0.33708507 0.43520991]
 [0.60309836 0.81425603 0.79111967]]
```

Come è possibile osservare nel caso precedente, i valori generati saranno tutti compresi tra 0 e 1.

È sufficiente moltiplicare i valori della matrice per spostare il range (nell'esempio seguente range 0:100).

```
import numpy as np
m_casuale = np.random.random((3,3))*100
print(m_casuale)
```

```
[[77.27403374 62.52783069 26.52249469]
 [56.3506733  21.18413943 69.35267354]
 [97.82611756 33.98919849 14.52111375]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per generare un array di valori tutti uguali ad un numero k, invece, abbiamo due opzioni:

- con la funzione ones() analizzata in precedenza e la successiva moltiplicazione per il valore k;
- con la funzione **full**() a cui si passa una tupla per le dimensioni e il valore k.

```
import numpy as np
m1 = np.ones((4,5),dtype = np.uint64)*7
m2 = np.full((4,5),7)
print("m1")
print(m1)
print()
print("m2")
print(m2)
```

```
m1
[[7 7 7 7 7]
 [7 7 7 7 7]
 [7 7 7 7 7]
 [7 7 7 7 7]]

m2
[[7 7 7 7 7]
 [7 7 7 7 7]
 [7 7 7 7 7]
 [7 7 7 7 7]]
```

È possibile, infine, inizializzare un array con valori uniformemente distribuiti in un intervallo. La funzione in questione è linspace() che richiede come argomenti i valori iniziale e finale dell'intervallo, oltre al numero di elementi che costituiranno l'array. Selezioniamo l'intervallo da 1 a 100 (estremi inclusi). Andiamo a creare tre array capaci di coprire l'intervallo con un numero diverso di elementi, rispettivamente 10, 5 e 7. Tutti gli array generati hanno in comune due fattori:

- il primo e l'ultimo valore equivalgono agli estremi dell'intervallo;
- tutti i valori che includono sono il più possibile equidistanti tra loro.

```
import numpy as np
a = np.linspace(1, 100, num = 10)
b = np.linspace(1, 100, num = 5)
c = np.linspace(1, 100, num = 7)
print("a = ",a)
print("b = ",b)
print("c = ",c)
```

```
a =  [  1.  12.  23.  34.  45.  56.  67.  78.  89. 100.]
b =  [  1.    25.75  50.5   75.25 100.  ]
c =  [  1.   17.5  34.   50.5  67.   83.5 100. ]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Indicizzazione e slicing

Gli array NumPy supportano le stesse operazioni di indicizzazione [i] e slicing [i:j] delle liste in Python. Gli array NumPy implementano, inoltre, l'indicizzazione booleana. È possibile, infatti, usare come indice un array di valori booleani. Il risultato della selezione sarà un array di quegli elementi dell'array di partenza per i quali l'indice booleano corrispondente è True. È possibile anche usare l'indicizzazione booleana sul lato destro (per una selezione) e sul lato sinistro (per un assegnamento).

Per mostrare le potenzialità dell'indicizzazione binaria si presenta un esempio di "pulizia" di un dataset. Ad esempio, si supponga di ricevere un dataset dirty e tutti i dati del dataset devono essere non negativi. Questo vuol dire che qualsiasi valore negativo del database non è un vero valore, ma un errore. Andremo, per puro scopo esemplificativo, a sostituire tali valori con uno zero. Si crea, quindi, un filtro booleano con la condizione dirty <0 e si utilizza, successivamente, tale filtro come indicizzazione dell'array per assegnare un nuovo valore.

```
import numpy as np
dirty = np.array([9, 4, 1, -0.01, -0.02, -0.001])
filtro_booleano = dirty <0
print(filtro_booleano)

dirty[filtro_booleano] = 0
print(dirty)
```

```
[False False False  True  True  True]
[9. 4. 1. 0. 0. 0.]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

È possibile combinare anche più espressioni booleane usando gli operatori | OR logico, & AND logico e – NOT logico.

Ad esempio, si vogliono conoscere gli elementi della seguente lista compresi tra -0.5 e +0.5. Andremo, quindi, ad utilizzare l'operatore & con le condizioni *lineare<=0.5* e *lineare >=-0.5*.

```
import numpy as np
lineare = np.arange(-1, 1.1, 0.2)
print(lineare)
print()
(lineare <= 0.5) & (lineare >= -0.5)
```

```
[-1.00000000e+00 -8.00000000e-01 -6.00000000e-01 -4.00000000e-01
 -2.00000000e-01 -2.22044605e-16  2.00000000e-01  4.00000000e-01
  6.00000000e-01  8.00000000e-01  1.00000000e+00]


array([False, False, False,  True,  True,  True,  True,  True, False,
       False, False])
```

Un importante aspetto da sottolineare è che:

- in **NumPy** gli operatori relazioni (es. <, ==) hanno precedenza inferiore rispetto agli operatori sui bit come & e | che rappresentano gli operatori booleani negli array numpy (ecco perché prima le condizioni inserite tra parentesi);
- in **Python** è il contrario - gli operatori booleani "normali" di Python or, and, not hanno precedenza inferiore rispetto agli operatori relazionali (solitamente, infatti, non si usano le parentesi tonde per raccogliere le condizioni).

Altre caratteristiche interessanti degli array NumPy sono l'**indicizzazione** e lo **slicing "intelligente"**. L'indice, in questo caso, non è uno scalare, ma un array o una lista di indici. Il risultato della selezione è l'array degli elementi individuati dall'indice.

Ad esempio, selezioniamo il secondo, il terzo e l'ultimo numero del vettore di 8 elementi (indicizzazione intelligente) come nel caso seguente:

```
import numpy as np
v = np.array(range(1,9))
print(v)
print()
s = v[[1,2,-1]]
print(s)
```

```
[1 2 3 4 5 6 7 8]

[2 3 8]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Proviamo, ora, ad effettuare uno slicing intelligente sulla seguente matrice.

```
[[1 2 3 4]
 [5 6 7 8]]
```

Estraiamo tutte le righe della colonna centrale. È possibile eseguire tale operazione in due modi (la decisione del modo è da prendere in base al risultato che si vuole ottenere anche in vista di operazioni successive):

- nel primo caso si utilizza per lo slicing [:,[1]] che restituisce una matrice di una riga e due colonne;

nel secondo caso per lo slicing si utilizza [:,1] che restituisce un array di due elementi.

```
import numpy as np
v = np.array(range(1,9))
m = v.reshape(2,4)
sm1 = m[:,[1]]
sm2 = m[:,1]
print("sm1=",sm1)
print("shape sm1",sm1.shape,"\n")
print("sm2=",sm2)
print("shape sm2",sm2.shape)
```

```
sm1= [[2]
 [6]]
shape sm1 (2, 1)

sm2= [2 6]
shape sm2 (2,)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5. Operazione tra array

Un aspetto molto interessante degli array NumPy è quella di poterli coinvolgere in operazioni cumulative tra di loro. Vediamo la somma di due array (elemento per elemento corrispondente) senza

```
import numpy as np
a = np.array([100, 25, 70, 55])
b = np.array([45, 12, 5, -21])

print(a+b)
```

```
[145  37  75  34]
```

l'utilizzo di cicli:

È possibile fare la stessa cosa con gli operatori di confronto. Il risultato (condizione b>0) sarà un array di un diverso tipo di dato (siamo passati da un array di interi ad uno di booleani):

```
import numpy as np
b = np.array([45, 12, 5, -21])
print(b>0)
```

```
[ True  True  True False]
```

Vediamo ora, il prodotto scalare di un vettore con un numero decimale. Si sottolinea che con Python l'asterisco indica l'operatore ripetizione, mentre in NumPy il **prodotto**.

```
import numpy as np
a = np.array([100, 25, 70, 55])
print(a*0.8)
```

```
[80. 20. 56. 44.]
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

5.1 Funzioni vettoriali universali

Le **funzioni vettoriali universali**, o **ufunc**, rappresentano la controparte a funzione delle operazioni vettoriali. È possibile applicare le ufunc a tutti gli elementi di un array in una sola chiamata a funzione.

Le ufunc offerte da NumPy sono:

- **aritmetiche** – add(), multiply(), negative(), exp(), log(), sqrt();
- **trigonometriche** – sin(), cos(), hypot();
- **bit-a-bit** – bitwise:and(), left_shift();
- **relazionali e logiche** – less(), logical_not(), equal();
- maximum() e minimum();
- **funzioni per numeri in virgola mobile** – isinf(), isfinite(), floor(), isnan().

Supponiamo di aver registrato in un array monodimensionale chiamato *azioni* le quotazioni delle azioni di borsa per otto simboli, prima e dopo il weekend:

```
import numpy as np
azioni = np.array([140.49, 0.97, 40.68, 41.53, 55.7, 57.21, 98.2,
                   99.19, 109.96, 111.47, 35.71, 36.27, 87.85, 89.11,
                   30.22, 30.91])
print(azioni)
```

```
[140.49   0.97  40.68  41.53  55.7   57.21  98.2   99.19 109.96 111.47
  35.71  36.27  87.85  89.11  30.22  30.91]
```

Si vogliono conoscere le quotazioni che hanno perso valore durante il weekend. Raggruppiamo, quindi, i prezzi in base al titolo e trasformiamo l'array di partenza in una matrice 2x8 con la funzione reshape().

```
import numpy as np
azioni = np.array([140.49, 0.97, 40.68, 41.53, 55.7, 57.21, 98.2,
                   99.19, 109.96, 111.47, 35.71, 36.27, 87.85, 89.11,
                   30.22, 30.91])
azioni = azioni.reshape(8,2).T
print(azioni)
```

```
[[140.49  40.68  55.7   98.2  109.96  35.71  87.85  30.22]
 [  0.97  41.53  57.21  99.19 111.47  36.27  89.11  30.91]]
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Ora è possibile applicare la funzione universale **greater**() a entrambe le righe svolgendo un confronto per colonne. Il risultato sarà un array di valori booleani.

```
import numpy as np
azioni = np.array([140.49, 0.97, 40.68, 41.53, 55.7, 57.21, 98.2,
                   99.19, 109.96, 111.47, 35.71, 36.27, 87.85, 89.11,
                   30.22, 30.91])
azioni = azioni.reshape(8,2).T
variaz = np.greater(azioni[0], azioni[1])
print(variaz)
```

```
[ True False False False False False False False]
```

Avendo l'array dei titoli (array di stringhe), è possibile utilizzare un'indicizzazione booleana per ottenere il titolo che ha perso il suo valore nel weekend.

```
import numpy as np
azioni = np.array([140.49, 0.97, 40.68, 41.53, 55.7, 57.21, 98.2,
                   99.19, 109.96, 111.47, 35.71, 36.27, 87.85, 89.11,
                   30.22, 30.91])
azioni = azioni.reshape(8,2).T
variaz = np.greater(azioni[0], azioni[1])
titoli = np.array(["MMM", "ABT", "ABBV", "ACN", "ACE", "ATVI", "ADBE", "ADT"])
print(titoli[variaz])
```

```
['MMM']
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Oltre ai numeri «tradizionali», NumPy supporta lo standard per numeri in virgola mobile IEEE 754 e offre i simboli per **l'infinito positivo** (inf) e per i **"non numeri"** (nan – not a number).

nan, per convenzione, si usa come segnaposto per i dati mancanti. La funzione universale **isnan**() è uno strumento ottimo, infatti, per individuare gli errori nei dati. Può essere utilizzato anche per la sostituzione dei dati mancanti con uno zero (operazione non sempre corretta dal punto di vista logico-funzionale).

```
import numpy as np
titoli = np.array(["MMM", "ABT", "ABBV", "ACN", "ACE", "ATVI", "ADBE", "ADT"])
azioni = np.array([140.49, np.nan, 40.68, 41.53, 55.7, 57.21, 98.2,
                   99.19, 109.96, 111.47, 35.71, 36.27, 87.85, 89.11,
                   30.22, 30.91])
azioni = azioni.reshape(8,2).T
print(np.isnan(azioni))
print()
azioni[np.isnan(azioni)]=0
print(azioni)
```

```
[[False False False False False False False False]
 [ True False False False False False False False]]

[[140.49  40.68  55.7   98.2  109.96  35.71  87.85  30.22]
 [  0.    41.53  57.21  99.19 111.47  36.27  89.11  30.91]]
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Zinoviev, D. (2017). Data science con Python. Apogeo.
- Maggi, G. (2020). Data science con Python. Edizioni LSWR.
- Marin, I., Shukla, A. Sarang, VK. (2019). L'analisi dei Big Data con Python. Tecniche nuove.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).